

Diseño e Implementación de un Analizador Léxico–Sintáctico para un Lenguaje Formal de Monitoreo de Incidentes Viales en el Puente General Belgrano

Fernando Arce Obregon; Hugo Fernando Elías; Gabriela Antonella Montanaro; Juan Cruz San Lorenzo Díaz; Tomás Agustín Villarreal

Facultad de Ciencias Exactas y Naturales y Agrimensura – UNNE, Teoría de la Computación,
fernandoarceobregon52@gmail.com

Facultad de Ciencias Exactas y Naturales y Agrimensura – UNNE, Teoría de la Computación,
hugoelias687@gmail.com

Facultad de Ciencias Exactas y Naturales y Agrimensura – UNNE, Teoría de la Computación,
gabrielaam201@gmail.com

Facultad de Ciencias Exactas y Naturales y Agrimensura – UNNE, Teoría de la Computación,
sanlorenzojuancruz@gmail.com

Facultad de Ciencias Exactas y Naturales y Agrimensura – UNNE, Teoría de la Computación,
tomivillarreal@gmail.com

Resumen

En este trabajo se presenta el diseño e implementación de un lenguaje formal orientado al registro estructurado de incidentes viales ocurridos en el Puente General Manuel Belgrano, corredor estratégico que conecta las provincias de Chaco y Corrientes. La propuesta busca transformar reportes informales en una representación sintácticamente válida y libre de ambigüedades, permitiendo su procesamiento automático por sistemas informáticos.

Para ello se definió una gramática libre de contexto expresada en notación BNF, capaz de modelar incidentes, alertas, acciones de respuesta, prioridades y ubicaciones. Asimismo, se desarrolló un analizador léxico encargado de reconocer los tokens del lenguaje y un parser descendente recursivo responsable de validar la estructura sintáctica de las entradas y detectar errores.

El sistema implementado genera un Árbol Sintáctico Abstracto (AST) que representa la estructura lógica de cada reporte procesado. Finalmente, se diseñó un conjunto de casos de prueba válidos e inválidos que permitió verificar el correcto funcionamiento del analizador y demostrar la viabilidad del enfoque propuesto para aplicaciones de monitoreo vial.

mediante mensajes informales o poco estructurados, la coordinación de las respuestas y la difusión de alertas pueden verse dificultadas.

1. Introducción

Los incidentes viales en el Puente General Manuel Belgrano representan una problemática cotidiana para las personas que transitan diariamente entre las provincias de Chaco y Corrientes. Situaciones como accidentes, vehículos averiados, congestiones o cortes parciales suelen generar importantes demoras y afectar la circulación durante largos períodos. Además, cuando estos eventos son comunicados

Con el propósito de contribuir a una gestión más eficiente de este tipo de situaciones, se diseñó un lenguaje formal especializado para describir y registrar incidentes ocurridos en el puente. El objetivo principal consiste en proporcionar una sintaxis uniforme que permita representar estos eventos junto con las acciones y alertas correspondientes, reduciendo ambigüedades y facilitando su procesamiento automático.

El proyecto aplica conceptos fundamentales de la Teoría de la Computación, particularmente gramáticas libres de contexto, análisis léxico y análisis sintáctico, demostrando cómo estas herramientas pueden utilizarse para modelar un problema real y desarrollar soluciones que favorezcan la estandarización y automatización de los reportes de tránsito.

2. Marco Teórico

Un lenguaje formal está compuesto por un conjunto de cadenas construidas siguiendo reglas sintácticas previamente definidas. Dichas reglas pueden representarse mediante gramáticas libres de contexto (GLC), ampliamente utilizadas en el diseño de compiladores e intérpretes.

El análisis léxico constituye la primera etapa del procesamiento y consiste en transformar una secuencia de caracteres en una lista de tokens significativos, eliminando espacios y detectando palabras reservadas, identificadores y símbolos.

Posteriormente, el análisis sintáctico verifica que la secuencia de tokens respete las producciones establecidas por la gramática. En este proyecto se implementó un parser descendente recursivo, seleccionado por la simplicidad de la gramática diseñada y por su facilidad de implementación mediante funciones recursivas.

Como resultado del análisis exitoso se genera un Árbol Sintáctico Abstracto (AST), estructura jerárquica que representa los elementos esenciales de la sentencia sin incluir detalles puramente sintácticos.

3. Diseño de la Gramática

La gramática fue definida utilizando notación BNF y pertenece a la clase de Gramáticas Libres de Contexto [1]. El lenguaje tiene como objetivo representar eventos relacionados con el monitoreo vial del Puente General Belgrano, permitiendo registrar incidentes, alertas, prioridades, ubicaciones y acciones recomendadas.

El símbolo inicial es *<Programa>*, que representa un bloque de registro compuesto por una lista de instrucciones. Para permitir el registro de múltiples eventos se incorporaron producciones recursivas en las listas de instrucciones, alertas y acciones.

Las principales producciones son las siguientes:

```

<Programa> ::= "REGISTRAR" "{"
<ListaInstrucciones> "}"

<ListaInstrucciones> ::= <Instruccion> |
<Instruccion> <ListaInstrucciones>

<Instruccion> ::= <ReporteIncidente> |
<ReporteAlerta>

<ReporteIncidente> ::= "INCIDENTE" ":"
IDENTIFICADOR "EN" IDENTIFICADOR
"PRIORIDAD" IDENTIFICADOR <Respuesta> ";"

<Respuesta> ::= <ListaAlertas>
<OpcionalAccionRespuesta> | <AccionRespuesta>
<OpcionalListaAlertas>

<OpcionalAccionRespuesta> ::= <AccionRespuesta>
| λ

<OpcionalListaAlertas> ::= <ListaAlertas> | λ

<AccionRespuesta> ::= "ACCION" ":"
<ListaAcciones>

<ListaAlertas> ::= <ReporteAlerta> |
<ReporteAlerta> "," <ListaAlertas>

<ListaAcciones> ::= IDENTIFICADOR |
IDENTIFICADOR "," <ListaAcciones>

<ReporteAlerta> ::= "ALERTA" ":"
IDENTIFICADOR "EN" IDENTIFICADOR

```

La estructura propuesta permite modelar de manera organizada los distintos eventos que pueden ocurrir en el entorno vial. La utilización de recursión en las listas posibilita registrar múltiples instrucciones, alertas y acciones dentro de un mismo programa. Además, la gramática fue refinada para minimizar ambigüedades y facilitar la implementación de un parser predictivo basado en descenso recursivo.

4. Implementación

El sistema fue desarrollado utilizando TypeScript sobre el entorno de ejecución Node.js. La elección de TypeScript permitió incorporar tipado estático a los tokens y a las estructuras del AST (Abstract Syntax Tree), mejorando la legibilidad del código y facilitando la detección temprana de errores durante el desarrollo.

4.1. Arquitectura del Compilador

El flujo de procesamiento del lenguaje se dividió de forma modular en tres etapas secuenciales de análisis:

- **Analizador Léxico (Lexer):** Se implementó mediante el reconocimiento de palabras reservadas, delimitadores sintácticos y patrones definidos. Los tokens identificados corresponden a palabras clave (REGISTRAR, INCIDENTE, ALERTA, ACCION, EN y PRIORIDAD), delimitadores ({, }, :, ,, ;) y un token genérico denominado IDENTIFICADOR. Al unificar los tipos de incidente, prioridades, ubicaciones y acciones bajo este token genérico, se evitó hardcodear listas cerradas de términos en el lexer. La salida consiste en una secuencia ordenada de tokens lista para el análisis sintáctico.

- **Analizador Sintáctico (Parser):** Se implementó mediante la técnica de descenso recursivo [2], donde cada símbolo no terminal de la gramática se representa con una función encargada de verificar la sintaxis y construir de forma paralela los nodos del AST.

-**Analizador Semántico:** Se incorporó un componente semántico que valida que los valores de los tokens IDENTIFICADOR correspondan a los catálogos válidos de incidentes, prioridades, ubicaciones y acciones, los cuales se cargan de manera dinámica desde un archivo de configuración externo (configuracion.json). Esto permite extender las capacidades del lenguaje sin necesidad de modificar el parser o la gramática.

4.2. Modelado del Árbol de Sintaxis Abstracta (AST)

La traducción dirigida por la sintaxis construye en memoria un AST que representa la estructura semántica de la entrada de manera limpia. En TypeScript, cada nodo del AST se implementó utilizando interfaces fuertemente tipadas que heredan de una base común, tal como se detalla en la **Fig. 1**.

```
export interface ProgramaNode extends ASTNode {
  tipo: 'Programa'
  instrucciones: InstruccionNode[]
}

export type InstruccionNode = ReporteIncidenteNode | ReporteAlertaNode

export interface ReporteIncidenteNode extends ASTNode {
  tipo: 'ReporteIncidente'
  tipoIncidente: string
  ubicacion: string
  prioridad: string
  respuesta: RespuestaNode
  linea: number
  columna: number
}

export interface ReporteAlertaNode extends ASTNode {
  tipo: 'ReporteAlerta'
  notificacion: string
  ubicacion: string
  linea: number
  columna: number
}

export interface RespuestaNode extends ASTNode {
  tipo: 'Respuesta'
  alertas: ReporteAlertaNode[] | null
  acciones: string[] | null
  orden: 'alertas_acciones' | 'acciones_alertas' | 'solo_alertas' | 'solo_acciones'
}
```

Fig. 1. Definición de tipos e interfaces del Árbol de Sintaxis Abstracta (AST) en TypeScript (ast.ts).

4.3. Representación Jerárquica Conceptual

Para comprender de manera integral cómo se organizan y relacionan estas estructuras en memoria, en la Figura 2 se presenta el modelo conceptual y jerárquico del Árbol de Sintaxis Abstracta (AST) diseñado para el lenguaje. Este esquema general ilustra la composición y el flujo de los datos desde el nodo raíz (ProgramaNode) hasta la derivación de sus componentes atómicos, evidenciando el desacoplamiento entre los reportes de incidentes complejos y las alertas autónomas, así como la estructura del plan de respuesta asociado a cada evento.

5. Casos de Prueba y Resultados

Casos de Prueba Válidos

Id-nombre del sistema: Sistema de Monitoreo Vial - Puente Gral. Belgrano.

Versión: 1.0. **Fecha:** 24/06/2026.

CP	Objetivo	Datos de Entrada	Resultado Esperado	Resultado Obtenido
1	Tokenización básica REGISTRAR y llaves en el lexer.	REGISTRAR { }	Lexer genera 4 tokens: REGISTRAR, LLAVE_ABRE, LLAVE_CIERRA, EOF.	Tokens generados: [REGISTRAR, LLAVE_ABRE, LLAVE_CIERRA, EOF]. El resultado esperado coincide y paso de manera exitosa.
2	Omisión de comentarios de línea en el lexer.	// Choque leve REGISTRAR { }	Omisión del comentario; mismos tokens que CP 1.	Tokens generados: [REGISTRAR, LLAVE_ABRE, LLAVE_CIERRA, EOF]. El lexer omitio con exito la linea de comentario.
3	Reporte básico con múltiples alertas y acciones en el parser.	REGISTRAR { INCIDENTE: accidente EN puente_sur PRIORIDAD alta ACCION llamar_policia, llamar_ambulancia ALERTA: desviar EN puente_sur; }	Parser valida sintaxis completa y punto y coma final.	El analisis sintactico fue correcto, procesando el bloque de instrucciones sin lanzar ninguna excepcion.
4	Alerta autónoma sin incidente asociado en el parser.	REGISTRAR { ALERTA: reducir_velocidad EN puente_norte; }	Acepta estructura simplificada sin INCIDENTE.	El analisis sintactico fue correcto, procesando exitosamente la alerta directa terminada en punto y coma.
5	Generación de AST jerárquico.	AST a partir del CP 3.	Nodo raíz con hijo ReporteIncidenteNode y listas completas.	Se estructuro un nodo Programa. El nodo ReporteIncidente tiene tipoIncidente: accidente, ubicacion: puente_sur, prioridad: alta. Su Respuesta contiene acciones: [llamar_policia, llamar_ambulancia], alertas: [desviar], orden: acciones_alertas.
6	Valores y ubicaciones en ReporteAlertaNode en el AST.	AST a partir del CP 4.	Propiedades notificación: reducir_velocidad y ubicación: puente_norte.	Se estructuro un nodo Programa. El nodo ReporteAlerta contiene

CP	Objetivo	Datos de Entrada	Resultado Esperado	Resultado Obtenido
				notificacion: reducir_velocidad y ubicacion: puente_norte de forma correcta.
7	Validar el reconocimiento de lexemas alfanuméricos (como las coordenadas km3 o km4) dentro del analizador léxico.	REGISTRAR { km3 }	Generación de secuencia de 5 tokens, incluyendo IDENTIFICADOR con valor literal km3.	Tokens generados: [REGISTRAR, LLAVE_ABRE, IDENTIFICADOR (km3), LLAVE_CIERRA, EOF]. Reconocimiento exitoso del identificador alfanumérico.
8	Evaluar el procesamiento sintáctico y modelado del AST ante una inversión del orden en el bloque de respuesta.	REGISTRAR { INCIDENTE: congestion EN acceso_corrientes PRIORIDAD media ALERTA: reducir_velocidad EN acceso_corrientes ACCION llamar_grua; }	Validación de gramática y estructuración jerárquica con orden alertas_acciones en el nodo Respuesta.	El análisis fue satisfactorio; se estructuró un AST con el orden alertas_acciones, integrando correctamente la notificación y la acción respectiva.

Casos de Prueba Inválidos

CP	Objetivo	Datos de Entrada	Resultado Esperado	Resultado Obtenido
9	Detección de caracteres inválidos en el lexer.	REGISTRAR { @ }	Generación de token ERROR con valor @.	Tokens generados: [REGISTRAR, LLAVE_ABRE, ERROR, LLAVE_CIERRA, EOF]. Se detecto el token de ERROR con valor arroba.
10	Falta de delimitación estructural (llaves) en el parser.	REGISTRAR ALERTA: desviar EN puente_sur;	Excepción sintáctica: se esperaba LLAVE_ABRE.	Fallo con el error sintactico: Error sintactico en linea 2, columna 3. Se esperaba LLAVE_ABRE, pero se encontro ALERTA.
11	Identificadores no registrados (Semántico).	REGISTRAR { INCIDENTE: ovni EN puente_sur PRIORIDAD alta; }	Excepción semántica: incidente 'ovni' no válido.	Fallo con el error semantico: Error semantico en linea 2, columna 14. El tipo de incidente 'ovni' no es valido.

CP	Objetivo	Datos de Entrada	Resultado Esperado	Resultado Obtenido
12	Validar la identificación de errores sintácticos ante la omisión del delimitador final (punto y coma) en un reporte de alerta en el parser.	REGISTRAR { ALERTA: desviar EN puente_sur }	El parser debe detectar la carencia del token PUNTO_COMA al procesar la LLAVE_CIERRA y emitir una excepción de sintaxis.	Fallo por error sintáctico: se esperaba PUNTO_COMA, pero se halló LLAVE_CIERRA (}). El comportamiento coincide con el análisis predictivo.

6. Discusión y Análisis

El desarrollo de la gramática se realizó de manera iterativa, incorporando observaciones surgidas durante las correcciones y pruebas realizadas a lo largo del proyecto. Este proceso de refinamiento permitió resolver desafíos tanto sintácticos como semánticos:

Compatibilidad LL(1) y Factorización de <Respuesta>: Originalmente, la producción <Respuesta> permitía redactar alertas y acciones en cualquier orden, pero su implementación directa generaba conflictos First/First bajo un lookahead de 1 (el parser no podía decidir de forma determinista qué producción derivar al encontrar tokens como ALERTA o ACCION). Para solucionar este inconveniente sin perder la flexibilidad del lenguaje, se aplicó una *factorización por la izquierda* utilizando derivaciones vacías (λ) en los no terminales auxiliares <OpcionalAccionRespuesta> y <OpcionalListaAlertas>. Esto eliminó matemáticamente las colisiones en los conjuntos de predicción léxica, garantizando que el compilador sea estrictamente LL(1).

Flexibilización de Terminales mediante el Analizador Semántico: En las primeras versiones, elementos específicos del dominio (como tipos de incidente, ubicaciones, niveles de prioridad y acciones) estaban hardcoded como tokens cerrados en el Lexer y la gramática. Esto limitaba la extensibilidad del lenguaje, ya que agregar una nueva ubicación requería modificar las reglas gramaticales y recompilar el analizador. La solución adoptada consistió en unificar estos elementos sintácticamente bajo el token genérico IDENTIFICADOR y trasladar su validación al *analizador semántico*. Al desacoplar estas listas y cargarlas dinámicamente desde un archivo configuracion.json, el lenguaje se volvió

altamente extensible y los mensajes de error pasaron a ser contextuales y precisos.

Redefinición de las Alertas e Integración de Ubicaciones: En las fases iniciales se trabajaba con notificaciones aisladas. Sin embargo, durante las pruebas prácticas se constató que una notificación sin un contexto espacial (por ejemplo, ordenar un desvío sin especificar dónde) resultaba insuficiente para la toma de decisiones en el puente. Para resolver este inconveniente, se reemplazó el esquema original por la producción <ReporteAlerta>, la cual integra de forma obligatoria tanto el tipo de notificación como la ubicación afectada ("ALERTA" ":" IDENTIFICADOR "EN" IDENTIFICADOR), aportando la precisión semántica requerida por el dominio vial.

Simplificación e Integridad del Lenguaje: La validación de múltiples ejemplos de uso facilitó la identificación y eliminación de reglas redundantes o excesivamente complejas. Este proceso iterativo dio como resultado una especificación gramatical final limpia, robusta y alineada con los requerimientos operativos reales de la infraestructura de monitoreo del Puente General Belgrano.

7. Conclusiones

En este trabajo se diseñó e implementó con éxito un lenguaje formal especializado en el registro y monitoreo de eventos viales para la gestión operativa del Puente General Belgrano. A partir de una Gramática Libre de Contexto en notación BNF, se desarrolló un compilador integrado por un analizador léxico, un parser predictivo por descenso recursivo y un validador semántico.

El proceso iterativo de diseño y las modificaciones de la entrega final permitieron extraer valiosas conclusiones técnicas y metodológicas:

- Rigurosidad y Determinismo: La aplicación de técnicas de factorización por la izquierda eliminó por completo los conflictos sintácticos presentes en las primeras versiones. Se demostró formalmente que la gramática obtenida es estrictamente de clase LL(1), lo que garantiza un análisis predictivo determinista con un rendimiento temporal lineal y un consumo de recursos sumamente eficiente.
- Extensibilidad y Separación de Incumbencias: El desacoplamiento de las listas de constantes del dominio (incidentes, prioridades, ubicaciones y acciones recomendadas) de la estructura interna del compilador demostró ser una decisión arquitectónica sobresaliente. Al unificar estos conceptos bajo un token genérico IDENTIFICADOR y delegar su validación a una capa semántica basada en un archivo de configuración JSON externo, se logró un lenguaje altamente mantenible que puede ampliarse operativamente en el tiempo sin necesidad de modificar el código fuente del parser.
- Generación de Estructura Semántica (AST): La implementación exitosa de la generación de un Árbol de Sintaxis Abstracta (AST) completamente tipado en TypeScript trasciende la mera validación de errores. El AST jerárquico estructurado constituye el nexo fundamental para la automatización de procesos viales, permitiendo la traducción directa de los reportes en flujos de trabajo ejecutables, visualizaciones en tableros de control y notificaciones automáticas a los servicios de emergencia.

En definitiva, se ha consolidado una solución formal y robusta que cumple con los estándares teóricos de la teoría de lenguajes y compiladores, sentando una base sólida para su integración en sistemas de gestión de tránsito inteligentes y futuras ampliaciones de la semántica del lenguaje.

Bibliografía

- [1] M. Sipser, Introduction to the Theory of Computation, 3rd ed., Boston, MA, EE. UU.: Cengage Learning, 2012.
- [2] R. M. y J. D. U. J. E. Hopcroft, Teoría de autómatas, lenguajes y computación, , 3.ª ed., Madrid, España: Pearson Educación, 2007.